PROGRAMMING & CS FUNDAMENTALS

Functional Programming Visual Guide

Purity, immutability, and functions as data — a calmer way to reason about code.

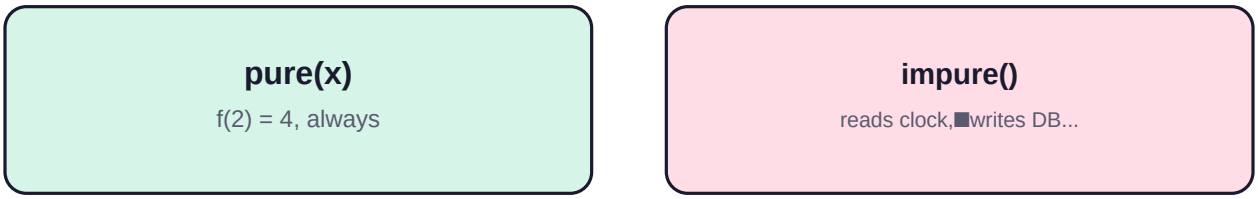
ANIMATED WALKTHROUGH

Step through pages to play the animation

STEP 1/6

Purity: Same Input, Same Output

No surprises: no hidden reads, no hidden writes.



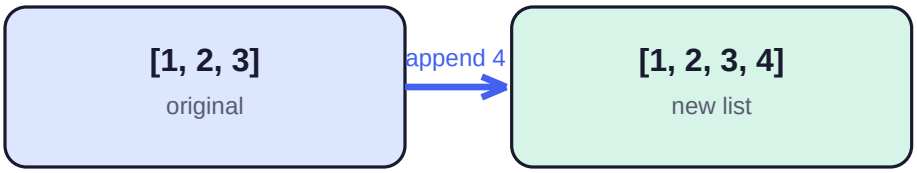

```
def pure_add(a, b): return a + b          # PURE: output from inputs only
total = 0
def impure_add(a):                       # IMPURE: mutates + reads global
    global total; total += a; return total
```

- Pure functions are trivially testable: no setup, no mocks, no teardown.
- Referential transparency: replace f(x) with its value anywhere — refactoring becomes safe.
- Push impurity to the edges: pure core, thin impure shell (I/O, randomness, time).

STEP 2/6

Immutability: Data That Never Changes

'Update' means 'derive a new version' — the old one survives.



original untouched — no aliasing bugs, safe to share across threads

```
# persistent structures share unchanged parts (structural sharing)
# tuple, frozenset, NamedTuple; dataclasses(frozen=True)
from dataclasses import dataclass
@dataclass(frozen=True)
class Point: x: int; y: int
```

- No defensive copies: share freely — nobody can mutate under you.
- Undo/history for free: keep old versions; time-travel debugging.
- Cost: allocation churn — persistent structures share structure to stay O(log n).

STEP 3/6

First-Class & Higher-Order Functions

Functions as values: passed, returned, composed.

```
def apply_twice(f, x): return f(f(x))      # takes a FUNCTION
apply_twice(lambda x: x + 1, 5)            # -> 7

def make_adder(n):                        # returns a FUNCTION
    return lambda x: x + n                 # closes over n
add5 = make_adder(5)
```

- Higher-order: takes or returns functions (map, filter, decorators, callbacks).
- Closures capture their defining scope — the adder remembers n=5.
- Composition: pipe small functions into pipelines — each stays testable alone.
- Callbacks, middleware, and hooks are higher-order functions in production clothes.

STEP 4/6

map / filter / reduce

The three verbs of collection processing.

```
nums = [1, 2, 3, 4, 5]
map:    [x*2 for x in nums]              # [2,4,6,8,10] transform each
filter: [x for x in nums if x % 2 == 0] # [2,4]         keep matches
reduce: functools.reduce(lambda a,b: a+b, nums)  # 15  fold to one
```

Pipeline style: read the data flow left to right, no loop bookkeeping.

```
# declarative pipeline: what, not how
total = sum(x*x for x in nums if x % 2 == 1)
```

- Each stage is pure and independent — trivially parallelizable (map-reduce!).
- Lazy variants (generators, streams) process infinite or huge data in O(1) memory.
- Prefer comprehensions/pipelines over manual loops for transforms — fewer index bugs.

STEP 5/6

Monads, Gently: The Maybe Pattern

Chain operations that might fail — without the if-pyramid.

```
# BEFORE: nested None checks
user = find(id)
if user: addr = user.address
...

# AFTER: Optional chaining / Maybe monad
city = find(id).map(get_address).map(get_city)  # None short-circuits
# Haskell: Maybe; Rust: Option; Python: optional chaining (3.10+ match)
```

- A monad = a wrapper (Maybe, Result, Future) + a way to chain (map/flatMap).
- Result types: return Ok(value) or Err(reason) — errors as VALUES, not exceptions.
- You already use monads: promises/futures chain async work exactly this way.
- Intuition over jargon: 'a box you can map over without opening it'.

STEP 6/6

Functional Traps

Purity has prices — know them.

- I/O is inherently impure — isolate it at the edges; don't fight it in the core.
- Deep recursion without TCO: Python will RecursionError — use loops or trampolines.
- Allocation churn: naive immutable updates copy O(n) — use persistent structures.
- Over-abstraction: point-free pipelines that nobody can read are a net loss.
- Not all state is evil: local, disciplined mutation inside a pure function is fine.
- Hybrid wins: functional core for logic, OO/imperative shell for structure and I/O.

STEP ✓I✓

Key Takeaways

Functional Programming — Remember

- Pure functions: same input, same output — testable, refactorable, parallelizable.
- Immutability: derive new versions instead of mutating; share freely, keep history.
- First-class functions enable higher-order patterns: map/filter/reduce, closures, composition.
- Monads (Maybe/Result/Future) chain fallible steps without nested checks.
- Push impurity to the edges: pure core, thin impure shell.

Keep exploring

Pair this guide with the matching video walkthroughs and hands-on labs at interstitiumlabs.dev